Dynamic programming solutions

15-E01 Rod-cutting revenue

Derivation and check

r0=0,r1=2; r2=max(5,2+2)=5; r3=max(7,2+5,5+2)=7; r4=max(10,2+7,5+5,7+2)=10; r5=max(12,2+10,5+7,7+5,10+2)=12. Optimum12 can be the uncut length5 or another tied cutting from substates. A fixed piece-count restriction needs a count dimension.

15-E02 Matrix-chain transition

Derivation and check

(A1A2)A3 costs2・3・4=24 then2・4・2=16, totaling40. A1(A2A3) costs3・4・2=24 then2・3・2=12, totaling36. Thus m[1,2]=m[2,3]=24 and m[1,3]=min(24+16,24+12)=36, split k=1, giving A1(A2A3).

15-E03 LCS transitions and reconstruction

Derivation and check

Columns ε,B,A,A and rows ε,A,B,A give[0,0,0,0],[0,0,1,1],[0,1,1,1],[0,1,2,2]. At(3,3), match A and move diagonally to(2,2). On that tie, left to(2,1) matches B, yielding reverse AB and result BA. Taking the tie upward to(1,2) instead matches A and yields AA. Both have optimal length2. Each move decreases a prefix index, proving termination.

15-E04 Global alignment states and reconstruction

Derivation and check

Let S[i,j] maximize the alignment score of X[1..i],Y[1..j]. Bases S[i,0]=−3i,S[0,j]=−3j. The recurrence maximizes diagonal plus3 on equality or−2 otherwise, up−3, and left−3. Pairing, X-terminal versus gap, and Y-terminal versus gap exhaust possible last columns. Removing that column leaves an optimum for shorter prefixes, or replacing it improves the original alignment.

Rows from ε through ACGTA, columns from ε through AGCAC are:

```
  0  -3  -6  -9 -12 -15
 -3   3   0  -3  -6  -9
 -6   0   1   3   0  -3
 -9  -3   3   0   1  -2
-12  -6   0   1  -2  -1
-15  -9  -3  -2   4   1
```

Dynamic programming solutions (continued)

From(5,5)=1 go left(5,4), diagonal(4,3), diagonal(3,2), diagonal(2,1), up(1,1), diagonal(0,0). Alignment is X′=ACGTA-,Y′=A-GCAC, with column scores3−3+3−2+3−3=1. Each input gains one gap. Equal-length-prefix comparisons miss insertion/deletion shifts; general alignment changes LCS's scoring model while retaining prefix DP and reconstruction.

15-E05 Complete unbounded-knapsack DP

Derivation and check

Let f[c] maximize exact-weight c value, with f[0]=0 and infeasible states−∞. $f[c] = \max_{i:s_i \le c}(f[c-s_i]+v_i)$; answer $\max_{0 \le c \le 13} f[c]$. If an optimum's last item is i, its predecessor at weight c−s_i must be optimal, since a better predecessor improves the original. Every candidate is feasible, proving completeness.

f[0..13]=[0,−∞,−∞,5,7,−∞,10,12,14,15,17,19,21,22]. Optimum22 occurs at13. Take final B and weight9=A×3, giving A+A+A+B. B has highest ratio7/4=1.75, but B×3 earns21, disproving general ratio greedy, not every possible greedy variant. Here at-most and exact13 coincide at22; allowing unused capacity still requires maximization across f[c]. Time Θ(SC), space O(C), for S item types and capacity C.

15-E06 Optimal BST with unequal failure probabilities

Derivation and check

Empty e[1,0]=.50,e[2,1]=.10,e[3,2]=.10. w[1,1]=.70 and e[1,1]=.50+.10+.70=1.30; w[2,2]=.40 and e[2,2]=.10+.10+.40=.60. w[1,2]=1. Root k1 costs.50+.60+1=2.10; root k2 costs1.30+.10+1=2.40. Thus root[1,2]=1; singleton roots are1 and2.

k1 has depth0,k2 depth1,d0 depth1,and d1/d2 depth2.
Cost=.10 • 1+.20 • 2+.50 • 2+.10 • 3+.10 • 3=2.10. Although p2>p1, the high-probability failure interval d0 makes k1 preferable. Real-key-only cost is2.10−(.50+.10+.10)=1.40, with the same optimal tree.

15-E07 Failure of endpoint-only longest-simple-path states

Derivation and check

u→z→v and v→z→w each are simple and length2. Concatenation u→z→v→z→w repeats z and is not simple. The actual simple u→w path is u→z→w, length2. The failure is shared internal vertices, not insufficiently long subsolutions.

Dynamic programming solutions (continued)

Let D[S,t] be the longest path from fixed u ending at t and using exactly set S. Base D[{u},u]=0; infeasible states−∞. For edges s→t with t∈S and s∈S excluding t, maximize D[S excluding t,s]+w(s,t). The added t is absent from the smaller state, preventing repetition. Removing the last vertex of any simple path gives a represented predecessor, proving completeness. Fill by increasing |S|. There are O(2^n n) states and O(2^n n^2) time by direct predecessor enumeration. Correctness is restored at exponential cost, not by a polynomial endpoint-only DP.

15-E08 Height, global weights, and two cost conventions

Outcome costs

Balanced success cost is0.50×2+0.10×1+0.05×2=1.20. Every failure compares two keys, giving0.35×2=0.70, hence C=1.90. The right chain has success cost0.50×1+0.10×2+0.05×3=0.85 and failure cost0.15×1+0.10×2+0.05×3+0.05×3=0.65, hence C=1.50. Its real-key height grows from1 to2 while expected cost falls.

Interval table and reconstruction

Interval	w	C	Root
[1,1]	0.75	0.75	k1
[2,2]	0.25	0.25	k2
[3,3]	0.15	0.15	k3
[1,2]	0.90	1.15	k1
[2,3]	0.35	0.50	k2
[1,3]	1.00	1.50	k1

For[1,3], candidate roots give0+0.50+1=1.50,0.75+0.15+1=1.90,and1.15+0+1=2.15. Choose k1, then root[2,3]=2 and root[3,3]=3, reconstructing the right chain. Global optimality follows from comparing all roots with optimal subtrees, not merely from comparing two specified shapes.

For every interval, E[i,j]=C[i,j]+sum(q[i−1..j]). Thus E[1,3]=1.85 and the balanced tree has E=2.25. Empty costs change from0 to q. The constant is the same for every shape of a given interval, preserving its optimal roots, including ties.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Dynamic programming solutions (continued)

Conditional costs need weights

C retains global probabilities, so child costs add directly. For conditional expectations $\bar{C}_L, \bar{C}_R$, first recover an unnormalized candidate:

$$C_r = p_r + W_L(1 + \bar{C}_L) + W_R(1 + \bar{C}_R) = w[i,j] + W_L\bar{C}_L + W_R\bar{C}_R.$$

Reweighting alone is insufficient: attachment adds W_L and W_R for one extra level, as well as the root's p_r. To keep a conditional parent expectation, also divide by w[i,j]. Do not divide a zero-mass interval by zero; unnormalized costs avoid the issue.

15-E09 Normalized weights and all optimal solutions

Complete interval table

Empty S[i,i−1]=0 and W[i,i−1]=Q[i−1]. Otherwise S is W plus the minimum child costs. These unnormalized weighted sums are not yet probability expectations.

Interval	W	S	Optimal roots
[1,1]	8	8	1
[2,2]	7	7	2
[3,3]	3	3	3
[4,4]	3	3	4
[1,2]	12	19	1
[2,3]	9	12	2
[3,4]	5	8	3 or4
[1,3]	14	25	2
[2,4]	11	19	2
[1,4]	16	32	2

Lin Cing https://github.com/xinhi6716/computer-science-self-study

The four full-interval candidates are16+(19,16,22,25)=(35,32,38,41). Root2 is uniquely optimal. Its left[1,1] has root1, but right[3,4] has two choices. The two optimum trees are k2(left k1,right k3(right k4)) and k2(left k1,right k4(left k3)). Attach d0..d4 to the corresponding missing-child gaps; both trees preserve key and gap order.

Total weight is16, so C=32/16=2. Failure weight is8, giving E=(32+8)/16=2.50. Positive scaling multiplies every tree's cost by1/16, preserving optima and ties.

Dynamic programming solutions (continued)

One root versus every optimum

c<best retains the first optimal root,3 in[3,4]. Changing to≤ retains the last,4, but still returns only one tree. To enumerate all optima, save all argmin roots and combine every optimal left/right subtree. Here there are exactly two trees. In general there can be exponentially many, so enumeration time must include the output cost.

Verification code appendix: dp_revision.py

```
"""Exact traces for the Chapter 15 revision pages."""

from fractions import Fraction
from itertools import combinations, product


def rod_cut(prices):
    n = len(prices) - 1
    revenue = [0] * (n + 1)
    choice = [0] * (n + 1)
    for length in range(1, n + 1):
        best_value = float("-inf")
        best_first = 0
        for first in range(1, length + 1):
            candidate = prices[first] + revenue[length - first]
            if candidate > best_value:
                best_value = candidate
                best_first = first
        revenue[length] = best_value
        choice[length] = best_first
    return revenue, choice


def rebuild_rod(choice, n):
    pieces = []
    while n:
        pieces.append(choice[n])
        n -= choice[n]
    return pieces


def matrix_chain(p):
    n = len(p) - 1
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Dynamic programming solutions (continued)

```
    inf = float("inf")
    m = [[0] * (n + 1) for _ in range(n + 1)]
    split = [[None] * (n + 1) for _ in range(n + 1)]
    for length in range(2, n + 1):
        for i in range(1, n - length + 2):
            j = i + length - 1
            m[i][j] = inf
            for k in range(i, j):
                candidate = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j]
                if candidate < m[i][j]:
                    m[i][j] = candidate
                    split[i][j] = k
    return m, split


def parenthesize(split, i, j):
    if i == j:
        return f"A{i}"
    k = split[i][j]
    return f"({parenthesize(split, i, k)}{parenthesize(split, k + 1, j)})"


def lcs_trace(x, y):
    n, m = len(x), len(y)
    c = [[0] * (m + 1) for _ in range(n + 1)]
    direction = [[""] * (m + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            if x[i - 1] == y[j - 1]:
                c[i][j] = c[i - 1][j - 1] + 1
                direction[i][j] = "diag"
            elif c[i - 1][j] >= c[i][j - 1]:
                c[i][j] = c[i - 1][j]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Dynamic programming solutions (continued)

```
                direction[i][j] = "up"
            else:
                c[i][j] = c[i][j - 1]
                direction[i][j] = "left"

    i, j = n, m
    result = []
    path = []
    while i and j:
        path.append((i, j, direction[i][j]))
        if direction[i][j] == "diag":
            result.append(x[i - 1])
            i -= 1
            j -= 1
        elif direction[i][j] == "up":
            i -= 1
        else:
            j -= 1
    return c, direction, "".join(reversed(result)), path


def optimal_bst(p, q):
    n = len(p)
    e = {}
    w = {}
    root = {}
    for i in range(1, n + 2):
        e[i, i - 1] = q[i - 1]
        w[i, i - 1] = q[i - 1]
    for length in range(1, n + 1):
        for i in range(1, n - length + 2):
            j = i + length - 1
            w[i, j] = w[i, j - 1] + p[j - 1] + q[j]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Dynamic programming solutions (continued)

```
            e[i, j] = None
            for r in range(i, j + 1):
                candidate = e[i, r - 1] + e[r + 1, j] + w[i, j]
                if e[i, j] is None or candidate < e[i, j]:
                    e[i, j] = candidate
                    root[i, j] = r
    return e, w, root


def global_alignment(x, y, match=3, mismatch=-2, gap=-3):
    score = [[0] * (len(y) + 1) for _ in range(len(x) + 1)]
    move = [[""] * (len(y) + 1) for _ in range(len(x) + 1)]
    for i in range(1, len(x) + 1):
        score[i][0] = i * gap
        move[i][0] = "up"
    for j in range(1, len(y) + 1):
        score[0][j] = j * gap
        move[0][j] = "left"
    for i in range(1, len(x) + 1):
        for j in range(1, len(y) + 1):
            diagonal = score[i - 1][j - 1] + (
                match if x[i - 1] == y[j - 1] else mismatch
            )
            up = score[i - 1][j] + gap
            left = score[i][j - 1] + gap
            score[i][j] = max(diagonal, up, left)
            move[i][j] = "diag" if diagonal == score[i][j] else "up" if up == score[i][j]
else "left"

    i, j = len(x), len(y)
    a, b = [], []
    while i or j:
        direction = move[i][j]
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Dynamic programming solutions (continued)

```
        if direction == "diag":
            a.append(x[i - 1])
            b.append(y[j - 1])
            i -= 1
            j -= 1
        elif direction == "up":
            a.append(x[i - 1])
            b.append("-")
            i -= 1
        else:
            a.append("-")
            b.append(y[j - 1])
            j -= 1
    return score, "".join(reversed(a)), "".join(reversed(b))


def unbounded_knapsack(items, capacity):
    best = [-1] * (capacity + 1)
    choose = [None] * (capacity + 1)
    best[0] = 0
    for cap in range(1, capacity + 1):
        for name, size, value in items:
            if size <= cap and best[cap - size] >= 0:
                candidate = best[cap - size] + value
                if candidate > best[cap]:
                    best[cap] = candidate
                    choose[cap] = name
    amount = max(range(capacity + 1), key=lambda cap: best[cap])
    chosen = []
    current = amount
    while current:
        name = choose[current]
        chosen.append(name)
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Dynamic programming solutions (continued)

```
        current -= next(size for item_name, size, _ in items if item_name == name)
    return best, amount, chosen


def audit_small_dp():
    lcs_cases = 0
    words = ["".join(chars) for n in range(5) for chars in product("AB",repeat=n)]
    for x in words:
        subsequences={"".join(x[i] for i in ids) for k in range(len(x)+1)
                      for ids in combinations(range(len(x)),k)}
        for y in words:
            other={"".join(y[i] for i in ids) for k in range(len(y)+1)
                   for ids in combinations(range(len(y)),k)}
            table, _, result, _ = lcs_trace(x,y)
            assert result in subsequences & other
            assert len(result)==table[-1][-1]==max(map(len,subsequences & other))
            lcs_cases+=1

    def all_costs(p,i,j):
        if i==j:
            return [0]
        return [left+right+p[i-1]*p[k]*p[j]
                for k in range(i,j)
                for left in all_costs(p,i,k)
                for right in all_costs(p,k+1,j)]

    matrix_cases = 0
    for dims in product((2,3),repeat=6):
        table,_=matrix_chain(dims)
        assert table[1][5]==min(all_costs(dims,1,5))
        matrix_cases+=1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dynamic programming solutions (continued)

```
    p=[Fraction(1,10),Fraction(2,10)]
    q=[Fraction(5,10),Fraction(1,10),Fraction(1,10)]
    cost,_,roots=optimal_bst(p,q)

direct=Fraction(1,10)*1+Fraction(2,10)*2+Fraction(5,10)*2+Fraction(1,10)*3+Fraction(1,10)*3
    assert roots[1,2]==1 and cost[1,2]==direct==Fraction(21,10)
    print(f"PASS independent_LCS_pairs={lcs_cases} enumerated_matrix_chains={matrix_cases}
high_q0_root={roots[1,2]} cost={direct}")


def main():
    empty_revenue, empty_choice = rod_cut([0])
    assert empty_revenue == [0] and rebuild_rod(empty_choice, 0) == []
    negative_revenue, negative_choice = rod_cut([0, -2, -5])
    assert negative_revenue == [0, -2, -4]
    assert rebuild_rod(negative_choice, 2) == [1, 1]
    rod_prices = [0, 1, 5, 8, 9]
    revenue, rod_choice = rod_cut(rod_prices)
    pieces = rebuild_rod(rod_choice, 4)
    assert revenue == [0, 1, 5, 8, 10]
    assert rod_choice == [0, 1, 2, 3, 2]
    assert pieces == [2, 2]

    p = [5, 10, 3, 12, 5]
    matrix_cost, split = matrix_chain(p)
    assert matrix_cost[1][4] == 405
    assert split[1][4] == 2
    assert parenthesize(split, 1, 4) == "((A1A2)(A3A4))"

    x, y = "ABCBDAB", "BDCABA"
    lcs_table, directions, lcs_value, lcs_path = lcs_trace(x, y)
    assert lcs_value == "BCBA" and lcs_table[-1][-1] == 4

    p_prob = [Fraction(1, 5), Fraction(3, 10), Fraction(1, 10)]
    q_prob = [
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Dynamic programming solutions (continued)

```
        Fraction(1, 10),
        Fraction(1, 20),
        Fraction(3, 20),
        Fraction(1, 10),
    ]
    bst_cost, bst_weight, bst_root = optimal_bst(p_prob, q_prob)
    assert bst_cost[1, 3] == Fraction(21, 10)
    assert bst_root[1, 3] == 2

    alignment_score, top, bottom = global_alignment("ACGTA", "AGCAC")
    assert alignment_score[-1][-1] == 1
    assert (top, bottom) == ("ACGTA-", "A-GCAC")

    items = [("A", 3, 5), ("B", 4, 7), ("C", 7, 12)]
    knapsack_values, amount, chosen = unbounded_knapsack(items, 13)
    assert knapsack_values[amount] == 22
    audit_small_dp()
    print(
        "PASS "
        f"rod={revenue} pieces={pieces} "
        f"matrix_cost={matrix_cost[1][4]} split={split[1][4]} "
        f"lcs={lcs_value} lcs_path={lcs_path} "
        f"bst={bst_cost[1, 3]} root={bst_root[1, 3]} "
        f"alignment={alignment_score[-1][-1]} {top}/{bottom} "
        f"knapsack={knapsack_values[amount]} capacity={amount} chosen={chosen}"
    )


if __name__ == "__main__":
    main()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dynamic programming solutions (continued)

Verification code appendix: longest_common_subsequence.py

```
"""Recover one LCS and verify it is a common subsequence."""

def lcs_table(x, y):
    table = [[0] * (len(y) + 1) for _ in range(len(x) + 1)]
    for i, xc in enumerate(x, 1):
        for j, yc in enumerate(y, 1):
            if xc == yc:
                table[i][j] = 1 + table[i - 1][j - 1]
            else:
                table[i][j] = max(table[i - 1][j], table[i][j - 1])
    return table


def rebuild(table, x, y):
    out, i, j = [], len(x), len(y)
    while i and j:
        if x[i - 1] == y[j - 1]:
            out.append(x[i - 1])
            i, j = i - 1, j - 1
        elif table[i - 1][j] >= table[i][j - 1]:
            i -= 1
        else:
            j -= 1
    return "".join(reversed(out))


table = lcs_table("ABCBDAB", "BDCABA")
candidate = rebuild(table, "ABCBDAB", "BDCABA")
assert table[-1][-1] == 4 and len(candidate) == 4
print(candidate)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dynamic programming solutions (continued)

Verification code appendix: rod_cutting.py

```
"""Bottom-up rod cutting with a reconstruction check."""

def rod_cut(prices):
    n = len(prices) - 1
    revenue = [0] * (n + 1)
    cut = [0] * (n + 1)
    for length in range(1, n + 1):
        candidates = [(prices[first] + revenue[length - first], first) for first in range(1,
length + 1)]
        revenue[length], cut[length] = max(candidates)
    return revenue, cut


prices = [0, 2, 5, 7, 10, 12]
revenue, cut = rod_cut(prices)
assert revenue[5] == 12
length, pieces = 5, []
while length:
    pieces.append(cut[length])
    length -= cut[length]
assert sum(prices[p] for p in pieces) == revenue[5]
print(revenue[5], pieces)
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study